

# Class 5: Practice 1 — Streamlit

Data-driven Systems Engineering

# Agenda

- What Streamlit is and why a data course needs it early.
- Installing Streamlit and running a first script.
- Reading the real dashboard entry point: 'streamlit/Main.py'.
- Where the fraud model behind that dashboard comes from: 'streamlit/modeling.py'.
- Hands-on practice: two small Streamlit applications.

## Learning goals

- Explain what Streamlit adds on top of a plain Python script.
- Install Streamlit and run an app locally.
- Read and modify an existing multi-page Streamlit application.
- Build a small Streamlit app from user input to computed output.
- Handle missing or invalid input with clear error and warning messages.

# Course Map and Running Cases



## Today in the course

**Focus:** First hands-on practice: turning a script into an interactive app with Streamlit.

**Bridge:** Loops, conditionals, functions, and lists from the last two classes now become the logic behind a page someone else can click through.

## Fraud case

The fraud case's dashboard, 'streamlit/Main.py' plus 'streamlit/pages/App\_Credit\_Card.py', is the app you will run today, not a hypothetical one.

## Pasteurization case

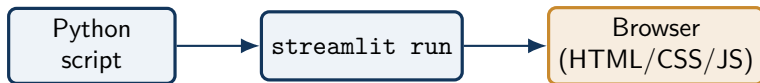
The same pattern later serves the pasteurization simulator: a Python script, wrapped in Streamlit, becomes a live monitoring page.

**Course thread:** From here on, “the app” in this course usually means a Streamlit app, and today is where that stops being abstract.

# What Streamlit is

- An open-source Python library for building interactive web apps, aimed at machine learning and data science.
- No HTML, CSS, or JavaScript required to get a working interface.
- A script is re-run top to bottom on every user interaction, which keeps the mental model simple.
- Good fit for demos, dashboards, and teaching; not a replacement for a full production frontend.

# How a Streamlit app is served



- You write plain Python; Streamlit's client-server layer renders it as a page at `localhost:8501` by default.
- Every widget interaction round-trips back to your script and re-executes it.

# Install and run

```
pip install streamlit  
streamlit run Main.py
```

- `pip install streamlit` adds it to your environment, alongside the other packages in 'requirements.txt'.
- `streamlit run <file>.py` launches the app and opens it in your default browser.
- Running it from inside 'streamlit/' starts the exact dashboard this course uses.

## Reading 'streamlit/Main.py'

```
st.set_page_config(  
    page_title="440MI - Machine Learning Dashboard",  
    layout="wide",  
    initial_sidebar_state="expanded"  
)  
  
st.title("Machine Learning Streamlit Hub")  
st.markdown("""  
Welcome to the 440MI + 305SM Course.  
This interface automatically discovers all available modules  
inside the 'pages/' directory.  
""")
```

- `set_page_config` runs once and configures the whole app; `title` and `markdown` render text on the page.
- Streamlit auto-discovers any script placed in `'streamlit/pages/'` as a separate page — that is how `'App_Credit_Card.py'` gets a sidebar entry with no extra wiring.

## Repo activity

- Run `streamlit run streamlit/Main.py` and open the sidebar to reach `App_Credit_Card`, `App_Pasteurization`, and `App_Stream`.
- Move a slider on the fraud page and watch the script re-run: this is the “re-run top to bottom on interaction” model from two slides ago, live.
- No new code is needed for this demo: the app already exists in this repository.

## Where the model behind the app comes from

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

cls = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
cls.fit(X_train_scaled, y_train)

with open('model.pkl', 'wb') as f:
    pickle.dump(cls, f)
```

- 'streamlit/modeling.py' is the script that produces model.pkl and scaler.pkl, the two files the app loads at start-up.
- You do not need to understand RandomForestClassifier yet — only that a trained object gets pickled to disk, exactly as any Python object can be.

## From pickle file to page

```
model = pickle.load(open('model.pkl', 'rb'))
scaler = pickle.load(open('scaler.pkl', 'rb'))

if st.button("Predict Fraud Risk"):
    prediction = model.predict(scaled)[0]
    probability = model.predict_proba(scaled)[0][1]
```

- 'App\_Credit\_Card.py' loads both pickled objects once, then reuses them on every button click.
- This closes the loop from Class 4: an if guarding a button click, calling a function (.predict) that returns a value.

## Practice task 1: descriptive statistics app

- Build a Streamlit app that accepts a list of float numbers from the user.
- Compute and display: mean, standard deviation, minimum, and maximum.
- Reuse the `summarize()` function pattern from Class 4 instead of writing the arithmetic inline.
- Keep the interface to a handful of widgets: one input, four displayed results.

## Practice task 2: student grouping app

- Input: the number of groups, and a list of student names.
- Output: the students organized into that many groups.
- Requirement: every group must have the same number of students.
- Handle the edge cases explicitly: an error message when there are no students, a warning when group sizes cannot come out even.

# A small widget cheat sheet

| Widget                                                                    | Typical use                                      |
|---------------------------------------------------------------------------|--------------------------------------------------|
| <code>st.number_input</code> / <code>st.slider</code>                     | Collect a numeric value from the user.           |
| <code>st.button</code>                                                    | Trigger an action, such as running a prediction. |
| <code>st.error</code> / <code>st.warning</code> / <code>st.success</code> | Give clear, colored feedback on the outcome.     |
| <code>st.title</code> / <code>st.markdown</code>                          | Static text and page structure.                  |

### Question

'App\_Credit\_Card.py' loads `model.pkl` once at the top of the script, even though the whole script re-runs on every interaction. Why does that matter for performance, and what would happen if loading were inside the button's `if` block instead?

- Introduces the idea that where code sits in a re-run model has real cost — a first, informal step toward the caching concepts used later in the course.

# Papers and materials

## Papers and materials

Get started with Streamlit; [Streamlit documentation](#); [scikit-learn documentation](#).

# From This Class to the Next

## What stays with us

- Streamlit turns a Python script into an interactive page with no separate frontend code.
- 'Main.py', 'modeling.py', and 'App\_Credit\_Card.py' already show the full path from training to pickle to interface.
- Practicing on two small apps builds the same muscle needed for the full fraud dashboard.

## Project use

If your project needs a quick interface for a stakeholder, Streamlit is very likely the fastest path there.

## Next connection

Class 6 goes back one step, to reading and writing files and plotting with pandas and Matplotlib, so the data behind these apps is not always hardcoded.

**Course thread:** Today's interface needs data; the next class is about getting that data in and visualized.